Transparent, wallet-controlled savings

- Enterprise Financial DApp · Sepolia prototype
- Zeran Yao · G2509029A · Sole contributor
- Python API · three roles · charts · fee preview · testnet only.

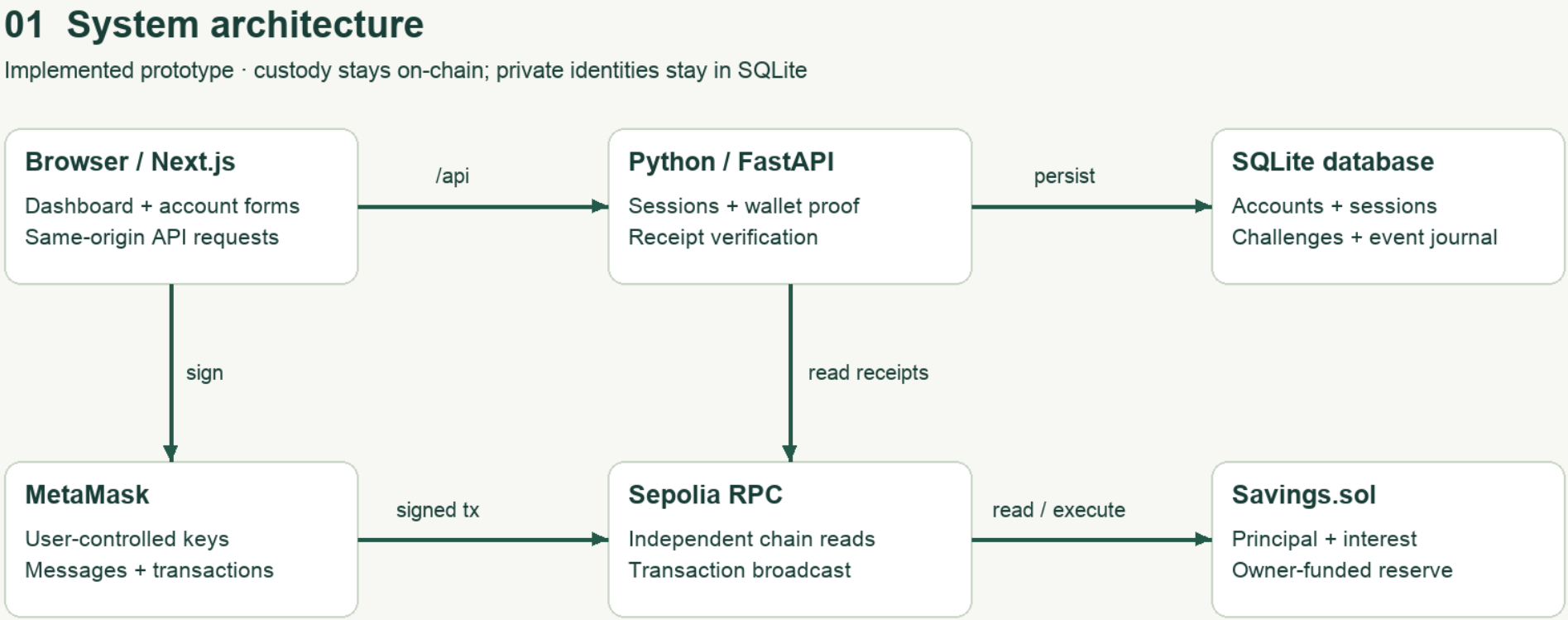
Can members verify the rules?

- Proposed setting: a crypto-literate membership savings incentive
- Member needs: visible principal, executable withdrawals and explainable rewards
- Sponsor funds interest; there is no external yield source
- Problem demand has not been validated with a real organisation.

Context supports investigation, not adoption claims

- World Bank Findex 2025: formal saving reached 40% of adults in developing economies in 2024 [1]
- This does not establish demand for a crypto savings app.
- BIS Papers 156 discusses DeFi asymmetry and stability risks [2].
- A bank or ordinary portal may be the better option for many users.

Identity and custody are separate



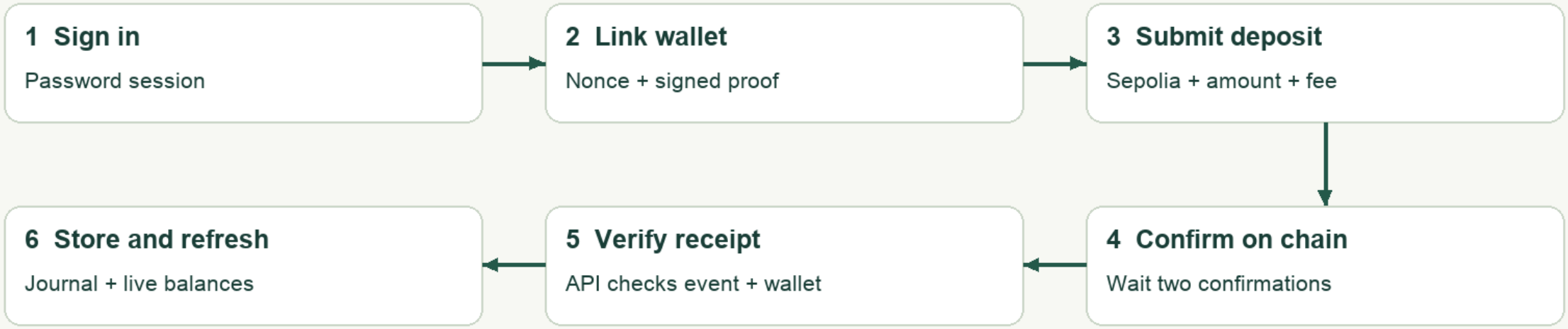
Four business transaction types

- Deposit → increase principal
- Withdraw → reduce principal and send ETH
- Claim → pay accrued interest from reserve
- Fund → owner supplies reserve; no admin access to member principal

Sign, confirm, verify, record

04 Wallet and deposit process

Message proof and an ETH transaction are separate approvals



On failure: retain hash, inspect explorer, retry import after confirmation. No fabricated success.

Local checks pass; scope is bounded

- Python: 16 cases, including migration, auditors, receipts and scoped history
- EVM: three tests covering reserve, accrual, top-ups and withdrawals
- Solidity compile, frontend lint and production build pass
- No live MetaMask, stakeholder pilot or independent security audit claimed.

Optimizer comparison, measured locally

- Deployment gas: 1017375 → 545131 (46.42% reduction)
- Creation bytecode: 4812 → 2366 bytes
- Optimized receipt gas: deposit 94735, fund 22724, claim 53951, withdraw 40698
- One controlled Ganache scenario; no current gas-price or latency claim

Principal protection is only one control

- Interest claims cannot consume principal
- Reentrancy guard and accounting before ETH transfer
- Salted password hashing, expiring cookies and one-use wallet challenges
- Open risks: finite reserve, lost wallet keys, RPC trust and missing recovery

Docker-ready; public evidence is pending

- Next.js standalone + Python/FastAPI + SQLite persistent disk
- Python runtime Docker build and browser-origin/session smoke passed
- No public Render URL, Sepolia address or GitHub history supplied
- Live deployment, restart persistence and performed demonstration still required.

Focused scope, transparent limits

- Implemented: Customer, Administrator and read-only Auditor
- Charting, paginated history, CSV, audit trail and fee preview
- One vault and four meaningful workflows retained under owner-authorised flexibility
- Public deployment, real MetaMask acceptance and performed demo remain pending

An inspectable prototype, with a clear next step

- Zeran Yao · G2509029A · 100% sole project contribution
- Artifacts: source, 14-page report, five diagrams, tests, manual and slides
- Next: finish mandatory workflows, deploy, validate with users and record live evidence
- No return, adoption or enterprise-readiness claim is made.